Studio Nx

Backend(FastAPI/Python) 81 — unexpected_error_handler() 수정

2026-10-12

Backend(FastAPI/Python) 81 — `unexpected_error_handler()` 수정 (500도 화면은 HTML)

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-22 오류 페이지 연결: 예상 못 한 서버 오류(500)를 화면 요청에는 오류 페이지로 응답하기 (PRD 01 FR-116)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	80 `http_error_handler()` · (P1-22 끝, 실제 화면 시험은 개요 문서에서)

1. 하는 일

버그나 DB 연결 끊김처럼 **예상하지 못한 오류**가 나면 500으로 응답하는 처리기에, 80번과 같은 화면/API 갈래를 더했습니다.

요청	전 (P1-03)	후 (P1-22)
API 주소에서 예상 못 한 오류	JSON 500 `INTERNAL_ERROR`	그대로
화면 주소에서 예상 못 한 오류	JSON 500	**`errors/500.html`을 상태 500으로**

어느 쪽이든 오류의 내용(SQL, 파일 경로, 비밀값이 섞일 수 있는 글자)은 응답에 넣지 않습니다.

2. 언제 쓰나

호출하는 곳	용도
`register_exception_handlers()`가 `Exception`에 연결	다른 처리기가 맡지 않은 모든 오류가 날 때

3. 코드

함수 (바뀐 부분)

```
async def unexpected_error_handler(request: Request, exc: Exception) -> Response:
    if is_api_request(request):
        return error_json(500, ErrorCode.INTERNAL_ERROR, "Something went wrong. Please try again.")
    return error_page(500)
```

바꾼 곳	내용
반환 형식	`JSONResponse`에서 `Response`로 (80번과 같은 이유)
갈래	API이면 예전 그대로 JSON, 화면이면 `error_page(500)`

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

4. 흐름

1. 요청 주소를 보고 API인지 판단합니다 (78번).
2. API이면 일반 문장의 JSON 500을, 화면이면 `errors/500.html`을 상태 500으로 돌려줍니다.
3. 응답을 보낸 뒤 Starlette가 원래 오류를 다시 던지므로, 서버 로그와 Sentry에는 원래 오류가 그대로 기록됩니다 (예전과 같습니다).

5. 왜 이렇게

선택	이유
화면에는 `error_page(500)`으로 고정	어떤 버그든 사용자에게는 같은 "서버 문제" 화면을 보여 주고, 내용은 로그로만 남깁니다
오류 내용을 HTML에 넣지 않음	파일 내용이 고정이라서 구조적으로 새지 않습니다
오류를 삼키지 않음	처리기는 응답만 바꾸고, 기록은 Starlette와 Sentry가 맡습니다 (06 설계 그대로)
기존 시험의 주소 `/bug`를 `/api/v1/bug`로 옮김	시험용 앱에서 API 응답(JSON)을 계속 확인하기 위해서입니다

6. 시험

`tests/test_errors.py`의 기존 2개는 주소만 `/api/v1/bug`로 옮겼고, `tests/test_http_error_pages.py`에 2개를 더 했습니다.

시험	결과
API 주소의 오류: 상태 500, 정해진 JSON, `password_hash`와 `RuntimeError` 글자 없음	통과입니다
API 주소의 오류도 응답 뒤에 다시 던져짐 (로그용)	통과입니다
화면 주소의 오류: 상태 500, HTML, `500`이 보임, 내부 글자 없음	통과입니다
화면 주소의 오류도 응답 뒤에 다시 던져짐 (로그용)	통과입니다

백엔드 pytest 전체 **568개 통과**, `ruff` 통과했습니다.